RaceDay

Part 1 – System Planning and Database

Module: PROGRAMMING 2B (PROG6212/w)
Assessment: Portfolio of Evidence (POE) – Part 1
Deliverables: Entity Relationship Diagram, API Endpoint Plan, SQL Database Script

GitHub Repository
github.com/ghost141-dev/RaceDay--st10485108-AS-MANNZWA-prog6212

Contents: Section A – Entity Relationship Diagram | Section B – API Endpoint Plan | Section C – SQL Database Script

Section A – Entity Relationship Diagram (ERD)

The diagram below shows the full RaceDay data model: six entities (Users, Events, Categories, Routes, Enrolments, Results), with primary keys, foreign keys, and the cardinality of every relationship. The SQL script in Section C matches this diagram exactly.

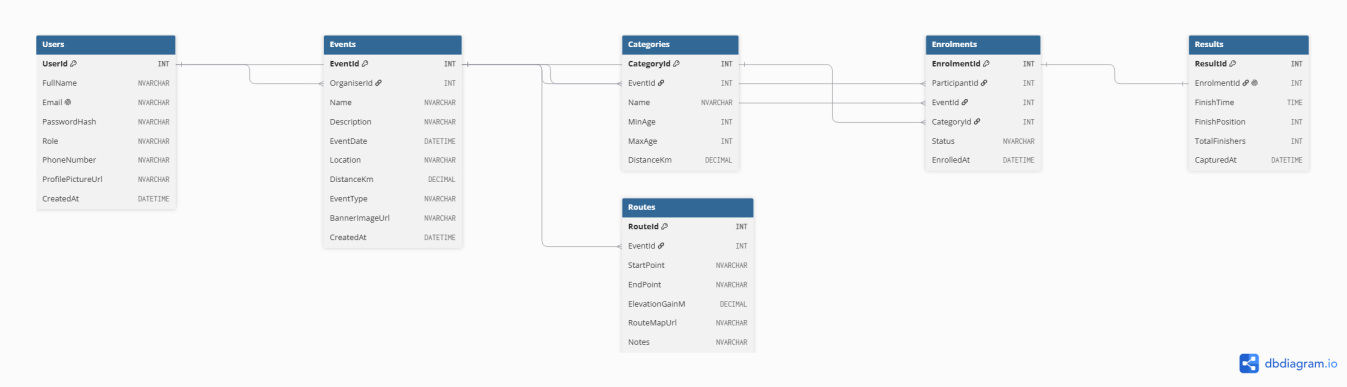


Figure A.1 — RaceDay Entity Relationship Diagram (dbdiagram.io)

Section B – API Endpoint Plan

Every endpoint the RaceDay API (built in Part 2) will expose is planned below. Role values: **None** (public, no login required), **Any** (any logged-in user, either role), **Organiser**, or **Participant**. All routes are prefixed with `/api/`. Non-public endpoints require an active session.

B.1 Authentication

Method	Route	Description	Role	Request Body	Expected Response
POST	`/api/auth/register`	Creates a new user account as either an Organiser or a Participant.	None	`{ fullName, email, password, role, phoneNumber }`	201 Created – user created (no password returned) 400 Bad Request – validation failed 409 Conflict – email already registered
POST	`/api/auth/login`	Authenticates a user and starts a session storing their user ID and role.	None	`{ email, password }`	200 OK – session started, returns basic user info and role 401 Unauthorized – invalid credentials
POST	`/api/auth/logout`	Ends the current user's session.	Any	`None`	200 OK – session ended 401 Unauthorized – no active session

B.2 User Profile

Method	Route	Description	Role	Request Body	Expected Response
GET	`/api/profile`	Returns the logged-in user's own profile information.	Any	`None`	200 OK – profile object 401 Unauthorized – not logged in
PUT	`/api/profile`	Updates the logged-in user's own profile details.	Any	`{ fullName, phoneNumber }`	200 OK – updated profile 400 Bad Request – validation failed 401 Unauthorized
POST	`/api/profile/picture`	Uploads or replaces the logged-in user's profile picture.	Any	`multipart/form-data (image file)`	200 OK – returns picture URL 400 Bad Request – invalid file type 401 Unauthorized

B.3 Events

Method	Route	Description	Role	Request Body	Expected Response
GET	`/api/events`	Lists all upcoming events, with optional filtering by type or location.	None	`None`	200 OK – array of events
GET	`/api/events/{id}`	Returns full detail for a single event, including its categories and route.	None	`None`	200 OK – event object 404 Not Found – event does not exist
POST	`/api/events`	Creates a new event owned by the logged-in Organiser.	Organiser	`{ name, description, eventDate, location, distanceKm, eventType }`	201 Created – new event 400 Bad Request – validation failed 401 Unauthorized 403 Forbidden – not an Organiser

Method	Route	Description	Role	Request Body	Expected Response
PUT	`/api/events/{id}`	Updates an event the logged-in Organiser owns.	Organiser	`{ name, description, eventDate, location, distanceKm, eventType }`	200 OK – updated event 403 Forbidden – not the owning Organiser 404 Not Found
DELETE	`/api/events/{id}`	Deletes an event the logged-in Organiser owns.	Organiser	`None`	204 No Content 403 Forbidden – not the owning Organiser 404 Not Found
POST	`/api/events/{id}/banner`	Uploads a banner image for the event.	Organiser	`multipart/form-data (image file)`	200 OK – returns banner URL 403 Forbidden – not the owning Organiser 404 Not Found

B.4 Categories

Method	Route	Description	Role	Request Body	Expected Response
GET	`/api/events/{eventId}/categories`	Lists all categories available for a specific event.	None	`None`	200 OK – array of categories 404 Not Found – event does not exist
POST	`/api/events/{eventId}/categories`	Adds a new age or distance category to an event the Organiser owns.	Organiser	`{ name, minAge, maxAge, distanceKm }`	201 Created – new category 403 Forbidden – not the owning Organiser 404 Not Found
PUT	`/api/categories/{id}`	Updates an existing category.	Organiser	`{ name, minAge, maxAge, distanceKm }`	200 OK – updated category 403 Forbidden 404 Not Found
DELETE	`/api/categories/{id}`	Removes a category from an event.	Organiser	`None`	204 No Content 403 Forbidden 404 Not Found 409 Conflict – category has existing enrolments

B.5 Event Enrolments

Method	Route	Description	Role	Request Body	Expected Response
POST	`/api/events/{eventId}/enrolments`	Enrols the logged-in Participant into an event under a chosen category.	Participant	`{ categoryId }`	201 Created – enrolment record with status 400 Bad Request – category does not belong to event 401 Unauthorized 403 Forbidden – not a Participant 409 Conflict – already enrolled in this event
GET	`/api/events/{eventId}/enrolments`	Lists all Participants enrolled in a specific event (for the owning Organiser).	Organiser	`None`	200 OK – array of enrolments 403 Forbidden – not the owning Organiser 404 Not Found
GET	`/api/enrolments/mine`	Lists all events the logged-in Participant has enrolled for, with status.	Participant	`None`	200 OK – array of the Participant's enrolments 401 Unauthorized
DELETE	`/api/enrolments/{id}`	Cancels the logged-in Participant's own enrolment before the event date.	Participant	`None`	204 No Content 403 Forbidden – not the owning Participant 404 Not Found 409 Conflict – event already concluded

B.6 Results

Method	Route	Description	Role	Request Body	Expected Response
POST	`/api/enrolments/{enrolmentId}/results`	Captures a finish time and position for a Participant's enrolment after the event.	Organiser	`{ finishTime, finishPosition, totalFinishers }`	201 Created – result recorded 400 Bad Request – validation failed 403 Forbidden – not the owning Organiser 404 Not Found 409 Conflict – result already captured
PUT	`/api/results/{id}`	Corrects a previously captured result.	Organiser	`{ finishTime, finishPosition, totalFinishers }`	200 OK – updated result 403 Forbidden 404 Not Found
GET	`/api/results/mine`	Returns the logged-in Participant's full personal race history.	Participant	`None`	200 OK – array of results across all completed events 401 Unauthorized
GET	`/api/events/{eventId}/results`	Returns all published results for an event (for the owning Organiser to review).	Organiser	`None`	200 OK – array of results 403 Forbidden – not the owning Organiser 404 Not Found

Section C – SQL Database Script

The script below creates the full RaceDay schema in SQL Server (via SSMS) and seeds it with realistic sample data: 2 Organisers, 2 Participants, 3 Events, categories per event, routes, enrolments, and results. It is safely re-runnable and matches the ERD in Section A exactly. The source file is committed at `/docs/RaceDay_Database.sql` in the GitHub repository linked on the cover page.

```
/* ============================================================
   RaceDay Database Script
   Module: PROG6212/w - Programming 2B
   Part 1: System Planning and Database

   This script creates the full RaceDay schema in SQL Server
   (via SSMS) and seeds it with realistic sample data.
   It matches the ERD stored in this /docs folder exactly.

   Run this script against a new, empty database. It has been
   tested to run cleanly from scratch (drops existing tables
   first so it is safely re-runnable during development).
   ============================================================ */

IF DB_ID('RaceDayDb') IS NULL
BEGIN
    CREATE DATABASE RaceDayDb;
END
GO

USE RaceDayDb;
GO

/* ------------------------------------------------------------
   Drop tables if they already exist (child tables first) so
   this script can be re-run cleanly during development.
   ------------------------------------------------------------ */
IF OBJECT_ID('dbo.Results', 'U') IS NOT NULL DROP TABLE dbo.Results;
IF OBJECT_ID('dbo.Enrolments', 'U') IS NOT NULL DROP TABLE dbo.Enrolments;
IF OBJECT_ID('dbo.Routes', 'U') IS NOT NULL DROP TABLE dbo.Routes;
IF OBJECT_ID('dbo.Categories', 'U') IS NOT NULL DROP TABLE dbo.Categories;
IF OBJECT_ID('dbo.Events', 'U') IS NOT NULL DROP TABLE dbo.Events;
IF OBJECT_ID('dbo.Users', 'U') IS NOT NULL DROP TABLE dbo.Users;
GO

/* ============================================================
   1. Users
   Stores both Organisers and Participants. The Role column
   distinguishes the two, avoiding duplicate account tables.
   ============================================================ */
CREATE TABLE dbo.Users
(
    UserId              INT IDENTITY(1,1)   NOT NULL,
    FullName            NVARCHAR(150)       NOT NULL,
    Email               NVARCHAR(150)       NOT NULL,
    PasswordHash        NVARCHAR(255)       NOT NULL,
    Role                NVARCHAR(20)        NOT NULL,
    PhoneNumber         NVARCHAR(20)        NULL,
    ProfilePictureUrl   NVARCHAR(500)       NULL,
    CreatedAt           DATETIME            NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_Users PRIMARY KEY (UserId),
    CONSTRAINT UQ_Users_Email UNIQUE (Email),
    CONSTRAINT CK_Users_Role CHECK (Role IN ('Organiser', 'Participant'))
);
GO

/* ============================================================
   2. Events
   Created and managed by an Organiser.
   ============================================================ */
```

```
CREATE TABLE dbo.Events
(
    EventId         INT IDENTITY(1,1)  NOT NULL,
    OrganiserId     INT                NOT NULL,
    Name            NVARCHAR(150)      NOT NULL,
    Description     NVARCHAR(1000)     NULL,
    EventDate       DATETIME           NOT NULL,
    Location        NVARCHAR(200)      NOT NULL,
    DistanceKm      DECIMAL(6,2)       NOT NULL,
    EventType       NVARCHAR(20)       NOT NULL,
    BannerImageUrl  NVARCHAR(500)      NULL,
    CreatedAt       DATETIME           NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_Events PRIMARY KEY (EventId),
    CONSTRAINT FK_Events_Organiser FOREIGN KEY (OrganiserId)
        REFERENCES dbo.Users (UserId),
    CONSTRAINT CK_Events_Type CHECK (EventType IN ('Run', 'Walk', 'Cycle'))
);
GO

/* ============================================================
   3. Categories
   Age or distance categories defined per event.
   ============================================================ */
CREATE TABLE dbo.Categories
(
    CategoryId  INT IDENTITY(1,1)  NOT NULL,
    EventId     INT                NOT NULL,
    Name        NVARCHAR(100)      NOT NULL,
    MinAge      INT                NULL,
    MaxAge      INT                NULL,
    DistanceKm  DECIMAL(6,2)       NULL,

    CONSTRAINT PK_Categories PRIMARY KEY (CategoryId),
    CONSTRAINT FK_Categories_Events FOREIGN KEY (EventId)
        REFERENCES dbo.Events (EventId)
);
GO

/* ============================================================
   4. Routes
   Route and elevation information participants use to prepare
   for race day. One event can have more than one route
   (e.g. a 10km route and a 21km route).
   ============================================================ */
CREATE TABLE dbo.Routes
(
    RouteId         INT IDENTITY(1,1)  NOT NULL,
    EventId         INT                NOT NULL,
    StartPoint      NVARCHAR(200)      NOT NULL,
    EndPoint        NVARCHAR(200)      NOT NULL,
    ElevationGainM  DECIMAL(6,2)       NULL,
    RouteMapUrl     NVARCHAR(500)      NULL,
    Notes           NVARCHAR(500)      NULL,

    CONSTRAINT PK_Routes PRIMARY KEY (RouteId),
    CONSTRAINT FK_Routes_Events FOREIGN KEY (EventId)
        REFERENCES dbo.Events (EventId)
);
GO

/* ============================================================
   5. Enrolments
   Links a Participant to an Event and the Category they chose.
   ============================================================ */
CREATE TABLE dbo.Enrolments
(
    EnrolmentId     INT IDENTITY(1,1)  NOT NULL,
    ParticipantId   INT                NOT NULL,
    EventId         INT                NOT NULL,
    CategoryId      INT                NOT NULL,
    Status          NVARCHAR(20)       NOT NULL DEFAULT 'Pending',
    EnrolledAt      DATETIME           NOT NULL DEFAULT GETDATE(),
```

```
    CONSTRAINT PK_Enrolments PRIMARY KEY (EnrolmentId),
    CONSTRAINT FK_Enrolments_Participant FOREIGN KEY (ParticipantId)
        REFERENCES dbo.Users (UserId),
    CONSTRAINT FK_Enrolments_Events FOREIGN KEY (EventId)
        REFERENCES dbo.Events (EventId),
    CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryId)
        REFERENCES dbo.Categories (CategoryId),
    CONSTRAINT CK_Enrolments_Status CHECK (Status IN ('Pending', 'Confirmed')),
    CONSTRAINT UQ_Enrolments_ParticipantEvent UNIQUE (ParticipantId, EventId)
);
GO

/* ============================================================
   6. Results
   Captured by the Organiser after the event concludes.
   One-to-one with Enrolments.
   ============================================================ */
CREATE TABLE dbo.Results
(
    ResultId        INT IDENTITY(1,1)  NOT NULL,
    EnrolmentId     INT                NOT NULL,
    FinishTime      TIME               NOT NULL,
    FinishPosition  INT                NOT NULL,
    TotalFinishers  INT                NOT NULL,
    CapturedAt      DATETIME           NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_Results PRIMARY KEY (ResultId),
    CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentId)
        REFERENCES dbo.Enrolments (EnrolmentId),
    CONSTRAINT UQ_Results_Enrolment UNIQUE (EnrolmentId)
);
GO

/* ============================================================
   SEED DATA
   2 Organisers, 2 Participants, 3 Events, categories per event,
   sample routes, enrolments and results.
   ============================================================ */

-- Users: 2 Organisers, 2 Participants
-- NOTE: PasswordHash values below are placeholder BCrypt-style
-- hashes for seed/demo purposes only, not real password hashes.
INSERT INTO dbo.Users (FullName, Email, PasswordHash, Role, PhoneNumber)
VALUES
('Thandiwe Nkosi',  'thandiwe.organiser@raceday.co.za', '$2a$11$SEEDHASH000000000000000000000000000000000000000000000
('Johan van der Merwe', 'johan.organiser@raceday.co.za',  '$2a$11$SEEDHASH000000000000000000000000000000000000000000000
('Lindiwe Dube',     'lindiwe.participant@raceday.co.za', '$2a$11$SEEDHASH000000000000000000000000000000000000000000000
('Sipho Mahlangu',   'sipho.participant@raceday.co.za',   '$2a$11$SEEDHASH000000000000000000000000000000000000000000000

-- Events (2 by Thandiwe (UserId 1), 1 by Johan (UserId 2))
INSERT INTO dbo.Events (OrganiserId, Name, Description, EventDate, Location, DistanceKm, EventType)
VALUES
(1, 'Joburg City 10K',      'A fast, flat 10km road run through the Johannesburg CBD.', '2026-03-14 07:00', 'Johar
(1, 'Soweto Community Walk', 'A family-friendly charity walk supporting local schools.', '2026-04-05 08:00', 'Sowe
(2, 'Cape Winelands Cycle Tour', 'A scenic road cycling tour through the Cape Winelands.', '2026-05-10 06:30', 'St

-- Categories per event
INSERT INTO dbo.Categories (EventId, Name, MinAge, MaxAge, DistanceKm)
VALUES
(1, '10km Open',      18,  NULL, 10.0),
(1, 'Under 20',       13,  19,   10.0),
(1, 'Senior (60+)',   60,  NULL, 10.0),
(2, '5km Family Walk', NULL, NULL, 5.0),
(3, '94.7km Individual', 18, NULL, 94.7),
(3, '94.7km Team Relay', 18, NULL, 94.7);

-- Routes
INSERT INTO dbo.Routes (EventId, StartPoint, EndPoint, ElevationGainM, Notes)
VALUES
(1, 'Mary Fitzgerald Square', 'Constitution Hill', 85.0,  'Mostly flat with one incline near the finish.'),
(2, 'Soweto Theatre',         'Freedom Square',    40.0,  'Suitable for wheelchairs and prams.'),
(3, 'Stellenbosch Square',    'Franschhoek Pass',  1120.0, 'Includes the Helshoogte Pass climb - bring water.');
-- Enrolments (Participants entering events)
```

```
INSERT INTO dbo.Enrolments (ParticipantId, EventId, CategoryId, Status)
VALUES
(3, 1, 1, 'Confirmed'),   -- Lindiwe entering Joburg City 10K, 10km Open
(4, 1, 2, 'Confirmed'),   -- Sipho entering Joburg City 10K, Under 20
(3, 3, 5, 'Pending');     -- Lindiwe entering Cape Winelands Cycle Tour

-- Results (captured after the event concludes)
INSERT INTO dbo.Results (EnrolmentId, FinishTime, FinishPosition, TotalFinishers)
VALUES
(1, '00:48:32', 47, 312),
(2, '00:52:10', 118, 312);
GO
```

End of Part 1 planning documents. Refer to the linked GitHub repository for the runnable .sql file, the ERD source, and the full commit history.